Test Assignment
Senior QA Engineer

Please track your actual time and include it in your submission. Include a brief write-up of your **test plan** and the next steps you would take to finish the assignment: which scenarios you would cover, how you would structure the remaining work, and any risks or open questions you would want to resolve. This gives us a clear picture of your thinking beyond what the code alone can show.

Goal

Write automated mobile tests for **one platform** of your choice, using our preferred tools:
Appium + WebdriverIO + TypeScript
XCUITest/Espresso / Compose UI tests
You **may** use other technologies, but in that case please include the motivation for why you chose them over the preferred ones.

The application under test is **Video QA Challenge** — a small, deterministic demo app that simulates a simplified media product: a consent screen, a video content overview, content detail pages, and a video player with an explicit, inspectable state machine. It requires no login, no network, and no external services.

- **OS**: https://github.com/tchumakina/video-qa-challenge-ios
- **Android**: https://github.com/tchumakina/video-qa-challenge-android

Build instructions are in each repository's README, and a prebuilt example binary is available in the `bin/` directory of each repository (a simulator `.app` for iOS, a debug APK for Android). The READMEs also document all test identifiers, launch configuration options, state reset mechanisms, and logging.

Scope

Suggested minimum scope:
1. Launch the application and handle the consent screen.
2. Open the video `Amsterdam from above` and verify the correct detail page is displayed.
3. Start video playback and verify the player reaches the `Playing` state.
4. Add at least one additional risk-based scenario of your choice.

You are free to go beyond this scope. The apps expose controllable loading, empty, and error states (switchable via debug options or launch configuration) that are well suited for

negative and edge-case testing. Loading delays are randomized within a small range by default and can be fixed to an exact value via launch configuration.

Deliverables

- A **public** GitHub/GitLab repository with your tests.
- A clear description of the solution and the **motivation** behind the chosen tools and approach.
- Instructions for how to run the tests.
- A test execution report stating on which environment the tests were executed (simulator/emulator, real device, device cloud, etc.) and why that environment was chosen.
- An AI usage note: describe whether and how you used AI tools (e.g. ChatGPT, Claude, Copilot) during the assignment.
If you did, include representative examples of the prompts you used — particularly any prompts you used to generate test scenarios, write test code, or reason about the application under test. There is no wrong answer; we are interested in how you leverage AI as a tool and how you evaluate its output.

What we look at

- Structure and readability of the test code.
- Reliability: stable locator strategy, proper synchronization (no arbitrary sleeps), deterministic state management between tests.
- Reasoning: why these tools, why these scenarios, why this environment.
- Quality of the report and documentation.